

Owen Manley

Professor Ghazaleh

CSC 450

Feb. 27th, 2025

Assignment #3

1. **Run process-run.py with the following flags: -l 5:100,5:100. What should the CPU utilization be (e.g., the percent of time the CPU is in use?) Why do you know this? Use the -c and -p flags to see if you were right. The CPU should consist of two processes with process 1 waiting for process 0 to complete to move forward with its utilization. Running “python3 process-run.py -l 5:100,5:100 -c” allowed me to see this process more closely. Through this command I saw how process 1 was put in the “READY” state until process 0 was “DONE”. In total, the process went through 10 clock ticks with 100% utilization from the CPU.**

```
Process 1
cpu
cpu
cpu
cpu
cpu
cpu

Important behaviors:
System will switch when the current process is FINISHED or ISSUES AN IO
After IOs, the process issuing the IO will run LATER (when it is its turn)

ubuntu@ubuntu:~/CSC450A3$ python3 process-run.py -l 5:100,5:100 -c
Time  PID: 0      PID: 1      CPU      IOs
1     RUN:cpu    READY      1
2     RUN:cpu    READY      1
3     RUN:cpu    READY      1
4     RUN:cpu    READY      1
5     RUN:cpu    READY      1
6     DONE      RUN:cpu    1
7     DONE      RUN:cpu    1
8     DONE      RUN:cpu    1
9     DONE      RUN:cpu    1
10    DONE      RUN:cpu    1

ubuntu@ubuntu:~/CSC450A3$ python3 process-run.py -l 5:100,5:100 -p
Produce a trace of what would happen when you run these processes:
Process 0
cpu
cpu
cpu
cpu
cpu

Process 1
cpu
cpu
cpu
cpu
cpu

Important behaviors:
System will switch when the current process is FINISHED or ISSUES AN IO
After IOs, the process issuing the IO will run LATER (when it is its turn)

ubuntu@ubuntu:~/CSC450A3$
```

2. Now run with these flags: `./process-run.py -l 4:100,1:0`. These flags specify one process with 4 instructions (all to use the CPU), and one that simply issues an I/O and waits for it to be done. How long does it take to complete both processes? Use `-c` and `-p` to find out if you were right. It took the CPU 6 clock ticks to complete its process while the I/O sat in the “READY” state. Then, it took the I/O 5 clock ticks because it had to set the “BLOCKED” state to the CPU so it would remain idle. In doing this, the CPU doesn’t interrupt or remain busy because I/O is a separate process aside from the CPU. The CPU was busy 54.55% of the time while I/O was busy for 45.45% of the time.

```
ubuntu@ubuntu: ~/CSC450A3
ubuntu@ubuntu:~/CSC450A3$ python3 process-run.py -l 4:100,1:0
Produce a trace of what would happen when you run these processes:
Process 0
cpu
cpu
cpu
cpu
cpu

Process 1
io
io_done

Important behaviors:
System will switch when the current process is FINISHED or ISSUES AN IO
After IOs, the process issuing the IO will run LATER (when it is its turn)

ubuntu@ubuntu:~/CSC450A3$ python3 process-run.py -l 4:100,1:0 -c
Time  PID: 0      PID: 1      CPU      IOs
1     RUN:cpu    READY      1
2     RUN:cpu    READY      1
3     RUN:cpu    READY      1
4     RUN:cpu    READY      1
5     DONE     RUN:io     1
6     DONE     BLOCKED    1
7     DONE     BLOCKED    1
8     DONE     BLOCKED    1
9     DONE     BLOCKED    1
10    DONE     BLOCKED    1
11*   DONE     RUN:io_done 1

ubuntu@ubuntu:~/CSC450A3$ python3 process-run.py -l 4:100,1:0 -p
Produce a trace of what would happen when you run these processes:
Process 0
cpu
cpu
cpu
cpu

Process 1
io
io_done

Important behaviors:
System will switch when the current process is FINISHED or ISSUES AN IO
After IOs, the process issuing the IO will run LATER (when it is its turn)
```

3. Switch the order of the processes:-l 1:0,4:100. What happens now? Does switching the order matter? Why? (As always, use-c and-p to see if you were right) Switching the order of processes did in fact make an impact on workload. Changing the order made the I/O process run about the same with 5 clock ticks, but it was much more busy at 71.43%. The CPU ran for the same time at 6 clock ticks and was much busier than before at 85.71%. Overall, the total time was 7 clock ticks in total which is better than the original order; however, they both were equally as busy and much more busy than what the order originally was. I think the case may be because the CPU was in the “BLOCKED” phase so it was idle. This didn’t allow the CPU to be in the “READY” state which caused it to work harder.

```
ubuntu@ubuntu: ~/CSC450A3
cpu
cpu
Important behaviors:
System will switch when the current process is FINISHED or ISSUES AN IO
After IOs, the process issuing the IO will run LATER (when it is its turn)

ubuntu@ubuntu:~/CSC450A3$ python3 process-run.py -l 1:0,4:100 -c
Time PID: 0 PID: 1 CPU IOs
1 RUN:io READY 1
2 BLOCKED RUN:cpu 1 1
3 BLOCKED RUN:cpu 1 1
4 BLOCKED RUN:cpu 1 1
5 BLOCKED RUN:cpu 1 1
6 BLOCKED DONE 1
7* RUN:io_done DONE 1
Stats: Total Time 7
Stats: CPU Busy 6 (85.71%)
Stats: IO Busy 5 (71.43%)

ubuntu@ubuntu:~/CSC450A3$ python3 process-run.py -l 1:0,4:100 -c -p
Time PID: 0 PID: 1 CPU IOs
1 RUN:io READY 1
2 BLOCKED RUN:cpu 1 1
3 BLOCKED RUN:cpu 1 1
4 BLOCKED RUN:cpu 1 1
5 BLOCKED RUN:cpu 1 1
6 BLOCKED DONE 1
7* RUN:io_done DONE 1
Stats: Total Time 7
Stats: CPU Busy 6 (85.71%)
Stats: IO Busy 5 (71.43%)

ubuntu@ubuntu:~/CSC450A3$ python3 process-run.py -l 1:0,4:100 -c -p
Time PID: 0 PID: 1 CPU IOs
1 RUN:io READY 1
2 BLOCKED RUN:cpu 1 1
3 BLOCKED RUN:cpu 1 1
4 BLOCKED RUN:cpu 1 1
5 BLOCKED RUN:cpu 1 1
6 BLOCKED DONE 1
7* RUN:io_done DONE 1
Stats: Total Time 7
Stats: CPU Busy 6 (85.71%)
Stats: IO Busy 5 (71.43%)

ubuntu@ubuntu:~/CSC450A3$
```

4. We'll now explore some of the other flags. One important flag is **-S**, which determines how the system reacts when a process issues an I/O. With the flag set to **SWITCH ON END**, the system will **NOT** switch to another process while one is doing I/O, instead waiting until the process is completely finished. What happens when you run the following two processes (**-l 1:0,4:100-c-S SWITCH ON END**), one doing I/O and the other doing CPU work? Using this flag allowed the process to finish the same time it did in its original form (before being flipped). Both the CPU and IO hadequal performance and clock time as well because of the flag that was set. I think through the exercises, I'm starting to see the difference. With the **-S** flag, the CPU was still Idle but process 1 was still able to be put in the "READY" state which is a significant difference between that and its initial iteration. Without the **-S** flagged, the process before was set to **SWITCH ON I/O** and didn't allow process 1 to prepare.

```
ubuntu@ubuntu: ~/CSC450A3
Time PID: 0 PID: 1 CPU IOs
1 RUN:io READY 1
2 BLOCKED READY 1
3 BLOCKED READY 1
4 BLOCKED READY 1
5 BLOCKED READY 1
6 BLOCKED READY 1
7* RUN:io_done READY 1
8 DONE RUN:cpu 1
9 DONE RUN:cpu 1
10 DONE RUN:cpu 1
11 DONE RUN:cpu 1
ubuntu@ubuntu:~/CSC450A3$ python3 process-run.py -l 1:0,4:100 -c -S SWITCH_ON_END -p
Time PID: 0 PID: 1 CPU IOs
1 RUN:io READY 1
2 BLOCKED READY 1
3 BLOCKED READY 1
4 BLOCKED READY 1
5 BLOCKED READY 1
6 BLOCKED READY 1
7* RUN:io_done READY 1
8 DONE RUN:cpu 1
9 DONE RUN:cpu 1
10 DONE RUN:cpu 1
11 DONE RUN:cpu 1
Stats: Total Time 11
Stats: CPU Busy 6 (54.55%)
Stats: IO Busy 5 (45.45%)
ubuntu@ubuntu:~/CSC450A3$ python3 process-run.py -l 1:0,4:100 -c -S SWITCH_ON_IO -p
Time PID: 0 PID: 1 CPU IOs
1 RUN:io READY 1
2 BLOCKED RUN:cpu 1 1
3 BLOCKED RUN:cpu 1 1
4 BLOCKED RUN:cpu 1 1
5 BLOCKED RUN:cpu 1 1
6 BLOCKED RUN:cpu 1 1
7* RUN:io_done DONE 1
Stats: Total Time 7
Stats: CPU Busy 6 (85.71%)
Stats: IO Busy 5 (71.43%)
ubuntu@ubuntu:~/CSC450A3$
```

In this exercise, I got to see first hand how the CPU handles different variations of processes using command prompts. More specifically, the command prompts allowed me to see the amount of time it takes for a process to finish, the steps it is taking, and other processes' steps (in this exercise the I.O). Overall, getting a visual representation of what we had been shown in class helps me in understanding the entire process a lot better.